Chapter 1 — Introduction to datasets

This part opens Chapter 1 by answering a deceptively simple question: what counts as a dataset?

Learning objectives

- Define a dataset as curated for analysis, not raw event streams
- Distinguish records from attributes (shared vocabulary)
- Same idea in CSV tables and nested JSON

What is a dataset?

- A dataset is a curated collection of data
- Has scope, structure, and documentation
- The dataset is the basic unit for ML, statistics, and BI

Records and attributes

- Each record is one observation; each attribute describes it
- The sales CSV uses Date, Product, Quantity, and Price as attributes
- The same mapping appears in healthcare and sensor examples later in the section

Example 1.1 — Sample CSV sales data

- Example 1.1 — hands-on module
- First concrete anchor: a small CSV of retail sales
- Each row is one sale; columns detail product, date, quantity, and price
- Explore the chapter example module
- View files: `modules/chapter1/example1/`

Example 1.1 — listing

```
Date,Product,Quantity,Price,Total
2024-01-01,Apple,3,0.5,1.5
2024-01-01,Banana,2,0.3,0.6
2024-01-02,Orange,5,0.7,3.5
2024-01-02,Grapes,1,2.0,2.0
2024-01-03,Mango,4,1.5,6.0
2024-01-04,Apple,2,0.5,1.0
```

Example 1.4 — Housing prices in JSON

- Example 1.4 — hands-on module
- Example 1.4 JSON listings with nested fields (flexible schema)
- The form changes; the objective stays the same: meaningful, analyzable data
- Explore the chapter example module
- View files: `modules/chapter1/example4/`

Example 1.4 — listing

```
[
  {
    "House_ID": 101,
    "Location": "New York",
    "Square_Footage": 2000,
    "Bedrooms": 3,
    "Proximity_to_Schools": "0.5 miles",
    "Price": 850000
  },
  {
    "House_ID": 102,
    "Location": "Los Angeles",
    "Square_Footage": 1800,
    "Bedrooms": 4,
    "Proximity_to_Schools": "1.2 miles",
    "Price": 720000
# ...
```

Takeaways

- A dataset is prepared for a purpose
- Records and attributes are the shared vocabulary
- Formats and modalities vary widely, but the goal of interpretable, usable data does not

Next

- Complete the quiz for this part
- Continue to the next part on types of datasets, structured, unstructured

Learning objectives

- Contrast the three structural types, name a typical format for each
- Explain when a dataset is static versus dynamic

Two classification axes

- Datasets are commonly classified along two axes
- Structure covers structured, unstructured, and semi-structured forms
- Temporal behavior covers static snapshots versus continuously updating streams
- Selecting an appropriate representation is a prerequisite for reliable analysis

Structured datasets

- Structured datasets follow a fixed schema: rows and columns with consistent types
- They live comfortably in spreadsheets and relational databases and are generally the
- A retail customer table

Unstructured datasets

- Unstructured data has no predefined row–column model
- Text, images, audio, and video fall here
- Valuable insights often require NLP or image recognition rather than a simple SQL filter
- Social media feedback and medical images are common examples

Semi-structured datasets

- Semi-structured data uses tags or keys
- Example 1.7 shows JSON objects where optional fields appear when relevant
- Open the example 7 module to inspect a concrete sample

Static vs dynamic

- Static datasets change rarely, an archived extract
- Dynamic datasets update continuously, sensor streams or click logs
- The same entity can move between forms over its lifecycle
- Temporal behavior shapes how often you refresh, version, and monitor the data

Takeaways

- Structure and time are independent
- Structured data is easiest to query

Next

- Complete the quiz for this part
- Complete the quiz, then continue to dataset formats, how these types are stored on disk

Learning objectives

- Leave able to name formats for tabular, hierarchical
- Geospatial data, explain when CSV, SQL, HDF5, or GeoJSON fits
- Connect format choice to the tools that will consume the data

Why format matters

- Format is the serialization of structure
- Matching format to structure, expected volume
- A mismatch, for example, forcing huge arrays through naive CSV, creates avoidable cost

Common formats at a glance

- CSV remains the common language for simple tables
- SQL tables add typed schemas and efficient queries
- JSON and GeoJSON handle nested objects and spatial features
- HDF5 targets large scientific or array-oriented datasets
- Each aligns with a structural niche

Example 1.8 — SQL format

- Example 1.8 — hands-on module
- Example 1.8 shows structured data in a relational form
- When attributes are stable and analysts need aggregations
- Explore the chapter example module
- View files: `modules/chapter1/example8/`

Example 1.8 — listing

```
]
CREATE TABLE Customers (
    customer_id SERIAL PRIMARY KEY,
    name VARCHAR(100) NOT NULL,
    email VARCHAR(100) UNIQUE NOT NULL
);

CREATE TABLE Orders (
    order_id SERIAL PRIMARY KEY,
    customer_id INT NOT NULL,
    order_date TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    total_amount DECIMAL(10,2),
    FOREIGN KEY (customer_id) REFERENCES Customers(customer_id)
);

CREATE TABLE Order_Items (
# ...
```

Example 1.9 — HDF5

- Example 1.9 — hands-on module
- Example 1.9 creates and reads an HDF5 file
- Scientific and machine-learning pipelines often prefer HDF5 when CSV becomes too slow or
- Explore the chapter example module
- View files: `modules/chapter1/example9/`

Example 1.9 — listing

```
import h5py
import numpy as np

with h5py.File("example.h5", "w") as hdf:
    data = np.random.rand(100, 100)
    hdf.create_dataset("random_data", data=data)
    group = hdf.create_group("experiment")
    group.create_dataset("temperature", data=np.linspace(0, 100, 100))
    group.create_dataset("pressure", data=np.linspace(1, 10, 100))

with h5py.File("example.h5", "r") as hdf:
    print("Datasets in root:", list(hdf.keys()))
    random_data = hdf["random_data"][:]
    print("Shape of 'random_data':", random_data.shape)
```

Choosing a format

- Start from structure, then consider size, update rate, and tooling
- Prefer formats your team already supports
- Metadata and documentation, covered next, make that choice durable

Takeaways

- Format serializes structure
- Everyday tables lean on CSV and SQL

Next

- Complete the quiz for this part
- Complete the quiz, then move to characteristics of good datasets, accuracy, completeness

Learning objectives

- Consistency, and explain why metadata and documentation make a dataset reusable

Five quality dimensions

- Table 1.6 in the chapter lists accuracy, completeness, consistency, relevance
- None stands alone
- Still on-question

Example 1.11 — Incorrect transaction

- Example 1.11 — hands-on module
- Example 1.11 shows a ledger where one transaction likely contains an extra zero
- That single accuracy error changes monthly revenue by an order of magnitude and can
- Explore the chapter example module
- View files: `modules/chapter1/example11/`

Example 1.11 — listing

```
Txn_ID,Date,Account,Amount,Type
T001,2024-03-01,Revenue,1500.00,Credit
T002,2024-03-02,Revenue,15000.00,Credit
T003,2024-03-03,Refund,50.00,Debit
```

Completeness and consistency

- Completeness failures
- Consistency failures
- Later chapters develop systematic repair; here the goal is recognition

Metadata and documentation

- Metadata records schema, units, provenance, and related context
- Example 1.14 illustrates metadata for a weather dataset
- Without these, even accurate tables become hard to trust outside the original team
- The example 14 module shows what good metadata looks like in practice

Takeaways

- Quality is multi-dimensional and task-relative
- Learn to spot accuracy errors, missing fields

Next

- Complete the quiz for this part
- Complete the quiz

Learning objectives

- A minimal profiling pattern using summaries, visuals, and missingness checks

Why explore first?

- Exploration builds a mental model of the domain
- It does not replace formal cleaning in later chapters

Three early questions

- Ask: what is in the table, what looks suspicious
- An e-commerce glance that reveals holiday spikes

Tools for exploration

- Spreadsheets suit small tables
- SQL handles filtering and aggregation in relational stores
- Pandas supports reproducible profiling
- Dashboards help teams share interactive views
- No single tool wins every scenario

Example 1.20 — Household incomes EDA

- Example 1.20 — hands-on module
- Example 1.20 walks through a standard pandas profiling script on a compact
- Use it as a template: summary statistics, distribution checks, and null counts
- Explore the chapter example module
- View files: `modules/chapter1/example20/`

Example 1.20 — listing

```
import pandas as pd
import matplotlib.pyplot as plt

# Load the dataset
household_incomes = pd.read_csv("household_incomes.csv")

# View the first few rows
print(household_incomes.head())

# Summary statistics for numerical columns
print(household_incomes.describe())

# Check for missing values
print(household_incomes.isnull().sum())

# Visualize the income distribution
# ...
```

Takeaways

- Explore before modeling
- Choose tools that fit size and structure
- Document what you inspected and what you decided to fix so later analysis stays

Next

- Complete the quiz for this part
- Complete the quiz

Learning objectives

- Map sectors to typical datasets and goals, recognize the shared lifecycle across domains
- Outline the steps in the customer-churn case study

Applications across sectors

- Use them to monitor performance, detect risk, or personalize services
- Retail emphasizes purchase and clickstream data

Example 1.21 — Purchase history

- Example 1.21 — hands-on module
- Example 1.21 stores one row per order with customer, category, date, and amount
- Aggregating by customer reveals repurchase intervals and category preferences that drive
- Explore the chapter example module
- View files: `modules/chapter1/example21/`

Case study: customer churn

- The churn case study stitches the chapter together
- Ranked feature importance highlights which behaviors precede churn

Example 1.31 — Churn model sketch

- Example 1.31 — hands-on module
- Example 1.31 provides a Python sketch of that pipeline
- The same structure appears in fraud scoring and demand forecasting
- Explore the chapter example module
- View files: `modules/chapter1/example31/`

Example 1.31 — listing

```
import pandas as pd
import numpy as np
from sklearn.ensemble import RandomForestClassifier
from sklearn.model_selection import train_test_split
from sklearn.impute import SimpleImputer
from sklearn.metrics import accuracy_score, precision_score,
recall_score, f1_score

# Sample dataset
data = {
    "Customer_ID": [1001, 1002, 1003, 1004, 1005, 1006, 1007, 1008,
1009, 1010],
    "Complaints": [2, 5, 8, 1, 3, 7, 0, 4, 6, 9],
    "Product_Usage": [50, 30, 20, 70, 45, 15, 80, 35, 25, 10],
    "Account_Activity_Change": [5, -10, -15, 8, -5, -20, 10, -7, -12,
-25],
    "Subscription_Length": [24, 36, 12, 48, 30, 6, 60, 28, 14, 10],
```

Takeaways

- Domain vocabulary changes; the lifecycle does not
- Quality and exploration feed every serious application
- Churn is the chapter's end-to-end illustration of define, prepare, model, and intervene

Next

- Complete the quiz for this part
- Complete the quiz, then finish the chapter with dataset management, versioning, access

Learning objectives

- Explain why management matters after a model ships
- Name privacy controls that protect sensitive extracts

Why management matters

- A dataset is collected, cleaned, documented, stored, shared, and sometimes retired
- Traceability links figures to their sources
- Without these foundations

Best practices (overview)

- Table 1.8 highlights version control
- These reinforce one another

Privacy and security

- Encryption, anonymization, role-based access with MFA, regulatory compliance
- Healthcare extracts, banking ledgers
- Ethical nuance continues in later chapters

Looking ahead

- Chapter 1 established definitions, types, quality, exploration, applications
- Chapter 2 turns to data collection

Takeaways

- Management is not optional
- Versioning, metadata, and access must work together
- Protection begins at first storage and continues through every reuse

Next

- Complete the quiz for this part
- Complete the quiz